MA563 HW4

Maggie Zhang (cz239)

2026-02-25

Problem 1: Discrete Least Squares

Let $x_j = jh$ with $h = \frac{2\pi}{N}$ (for $j = 0, \ldots, N-1$) be the usual nodes for the DFT.

---

1. Verify the Discrete Orthogonality Relation

We need to show that $\langle e^{ijx}, e^{ikx} \rangle_d = 0$ for $j \neq k$, where $\langle f, g \rangle_d = \sum_{j=0}^{N-1} f(x_j)\overline{g(x_j)}$.

Sol:

Compute:

$$\langle e^{ijx}, e^{ikx} \rangle_d = \sum_{\ell=0}^{N-1} e^{ijx_\ell}\, \overline{e^{ikx_\ell}} = \sum_{\ell=0}^{N-1} e^{ij\ell h} e^{-ik\ell h} = \sum_{\ell=0}^{N-1} e^{i(j-k)\ell h}.$$

Let $d = j - k$. Then the sum is a geometric series:

$$S = \sum_{\ell=0}^{N-1} \left(e^{idh}\right)^{\ell}.$$

Since $h = 2\pi/N$, we have $e^{idhN} = e^{id2\pi} = 1$.

- If d is a multiple of N, then $e^{idh} = 1$ and each term equals 1, so $S = N$.
- Otherwise, $e^{idh} \neq 1$ and the geometric series formula gives

$$S = \frac{1 - e^{idhN}}{1 - e^{idh}} = \frac{1 - 1}{1 - e^{idh}} = 0.$$

Thus for $j \neq k$ with $|j - k|$ not a multiple of N, the inner product is zero. In particular, for frequencies in the range $[-m, m]$ with $m < N/2$, this condition holds, so the exponentials are orthogonal on the discrete set $\{x_j\}$.

---

2. Least Squares Approximation

We are given values $f_j = f(x_j)$ at the nodes and seek an approximation

$$p_m(x) = \sum_{k=-m}^{m} c_k e^{ikx}, \quad m < \frac{N}{2},$$

that minimizes the discrete least squares error

$$E(\vec{c}) = \sum_{j=0}^{N-1} \left| p_m(x_j) - f_j \right|^2.$$

Interpret the data as a vector $\mathbf{f} = (f_0, \ldots, f_{N-1})^T \in \mathbb{C}^N$ and the basis functions as vectors

$$\boldsymbol{\phi}_k = \left(e^{ikx_0}, e^{ikx_1}, \ldots, e^{ikx_{N-1}}\right)^T, \quad k = -m, \ldots, m.$$

From part 1, these vectors are orthogonal with respect to the standard Hermitian inner product on $\mathbb{C}^N$:

$$\langle \boldsymbol{\phi}_k, \boldsymbol{\phi}_\ell \rangle = \sum_{j=0}^{N-1} e^{ikx_j} \overline{e^{i\ell x_j}} = 0 \quad (k \neq \ell),$$

and their squared norms are

$$\|\boldsymbol{\phi}_k\|^2 = \sum_{j=0}^{N-1} 1 = N.$$

The error E is the squared distance between $\mathbf{f}$ and the linear combination $\sum_k c_k \boldsymbol{\phi}_k$. In a finite-dimensional inner product space, the best approximation (minimizing the distance) is given by the orthogonal projection onto the subspace spanned by $\{\boldsymbol{\phi}_k\}$. The coefficients satisfy the normal equations

$$\sum_{\ell=-m}^{m} c_\ell \langle \boldsymbol{\phi}_\ell, \boldsymbol{\phi}_k \rangle = \langle \mathbf{f}, \boldsymbol{\phi}_k \rangle \quad \text{for each } k.$$

Due to orthogonality, this reduces to

$$c_k \langle \boldsymbol{\phi}_k, \boldsymbol{\phi}_k \rangle = \langle \mathbf{f}, \boldsymbol{\phi}_k \rangle,$$

so

$$c_k = \frac{\langle \mathbf{f}, \boldsymbol{\phi}_k \rangle}{\|\boldsymbol{\phi}_k\|^2} = \frac{1}{N} \sum_{j=0}^{N-1} f_j \, \overline{e^{ikx_j}} = \frac{1}{N} \sum_{j=0}^{N-1} f_j \, e^{-ikx_j}.$$

This is precisely the (scaled) discrete Fourier transform coefficient for frequency k. Thus the least squares solution yields the DFT coefficients.

Problem 2: Testing of Discrete Fourier Transform

(a) Fourier Series Approximation

We write a function that computes the partial sum S_N of the Fourier series for a given function f on $[0, 2\pi]$, then apply it to $f(x) = x$ and $f(x) = \sin(2x)$, plot $S_N(x)$, and plot the L^2 error $\|S_N - f\|_{L^2}$ versus N.

Fourier series used:

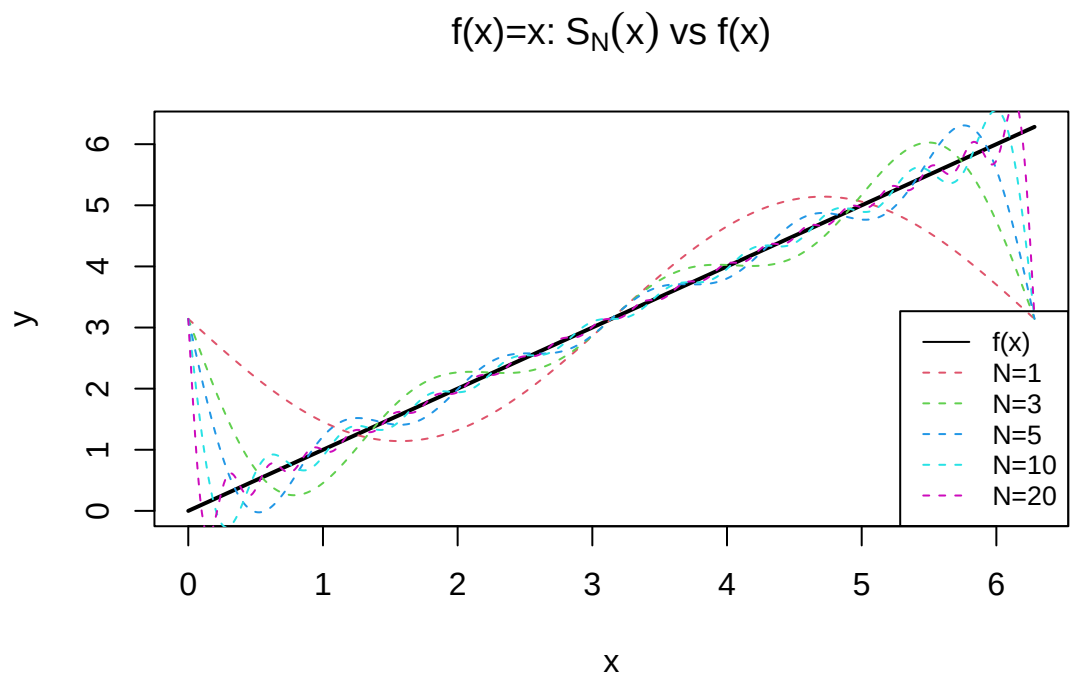
- For $f(x) = x$ on $[0, 2\pi]$: $a_0/2 = \pi$, $b_n = -\frac{2}{n}$ for $n \geq 1$, so $S_N(x) = \pi - \sum_{n=1}^{N} \frac{2}{n} \sin(nx)$.

- For $f(x) = \sin(2x)$: the Fourier series has only the term $\sin(2x)$, so $S_N(x) = \sin(2x)$ for $N \geq 2$.

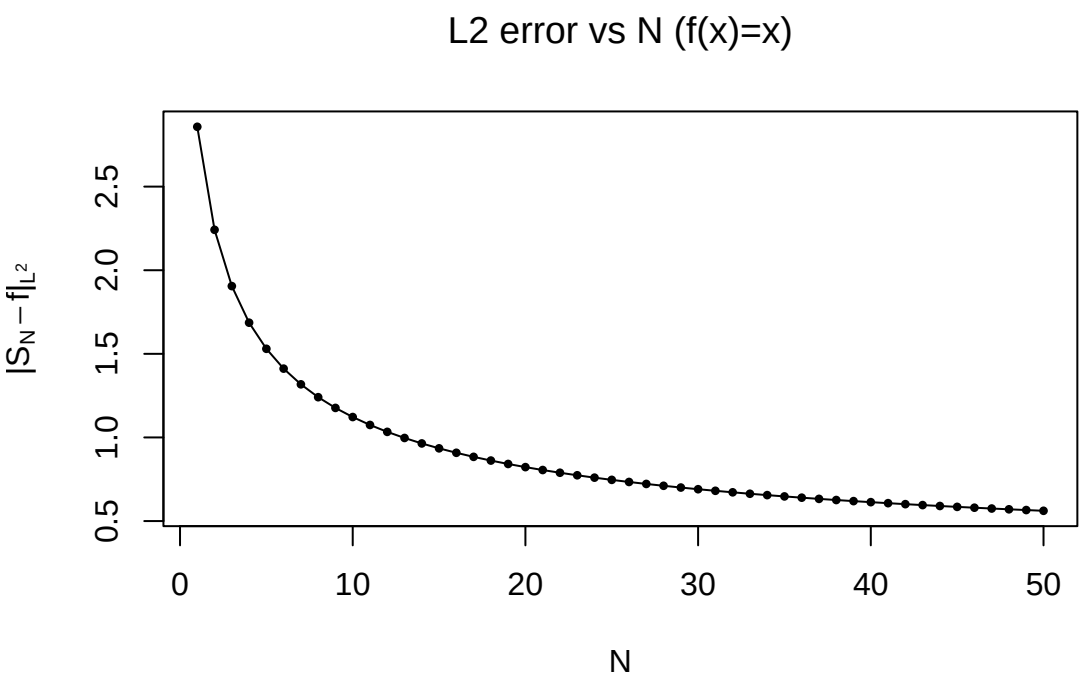
```r
# Partial sum S_N of Fourier series for f(x) = x on [0, 2*pi]
# S_N(x) = pi - sum_{n=1}^N (2/n) sin(n x)
S_N_x <- function(x, N) {
  if (N < 1) return(rep(pi, length(x)))
  s <- pi
  for (n in 1:N) s <- s - (2 / n) * sin(n * x)
  s
}

# Partial sum for f(x) = sin(2x): just sin(2x) for N >= 2
S_N_sin2x <- function(x, N) {
  if (N < 2) {
    # N=0: zero, N=1: only sin(x) coefficient would be 0 for sin(2x)
    return(rep(0, length(x)))
  }
  sin(2 * x)
}

# L2 norm of (g - f) on [0, 2*pi]: sqrt(integral_0^{2pi} |g(x)-f(x)|^2 dx)
L2_error <- function(g_fun, f_fun, N_vals, x_fun = S_N_x, n_points = 1000) {
  x <- seq(0, 2*pi, length.out = n_points)
  dx <- (2*pi) / (n_points - 1)
  err <- numeric(length(N_vals))
  for (i in seq_along(N_vals)) {
    N <- N_vals[i]
    g <- x_fun(x, N)
    f <- f_fun(x)
    err[i] <- sqrt(sum(abs(g - f)^2) * dx)
  }
  err
}
```

```r
# f(x) = x
x_plot <- seq(0, 2*pi, length.out = 500)
N_vals_plot <- c(1, 3, 5, 10, 20)

# Graph 1: S_N(x) vs f(x) = x
plot(x_plot, x_plot, type = "l", lwd = 2, col = "black",
     xlab = "x", ylab = "y", main = expression("f(x)=x:"~S[N](x)~"vs f(x)"))

# Adding the Fourier series partial sums
for (i in 1:length(N_vals_plot)) {
  N <- N_vals_plot[i]
  lines(x_plot, S_N_x(x_plot, N), col = i + 1, lty = 2)
}

legend("bottomright", legend = c("f(x)", paste0("N=", N_vals_plot)),
       col = c("black", 2:(length(N_vals_plot)+1)),
       lty = c(1, rep(2, length(N_vals_plot))),
```

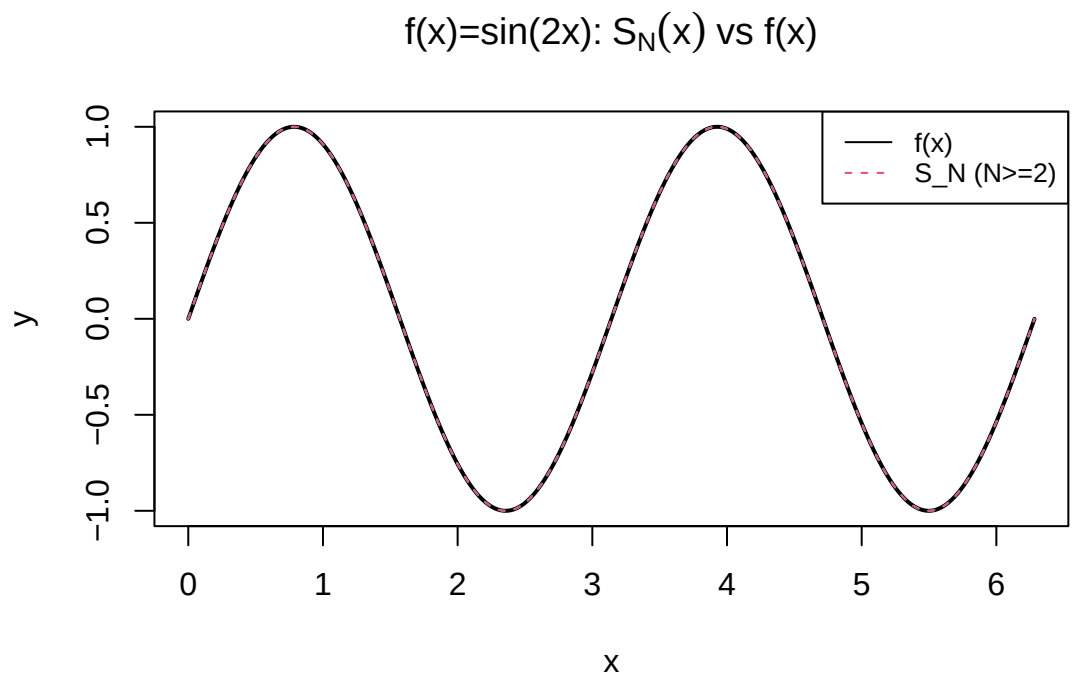
```
       cex = 0.8)
```



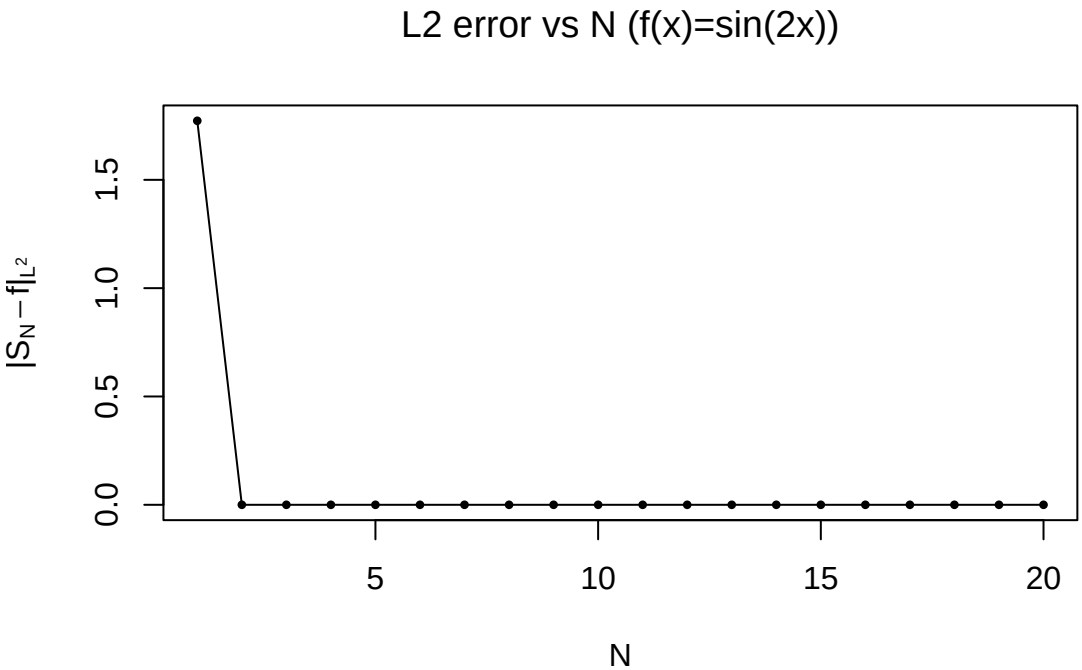

```
# Graph 2: L2 error vs N for f(x)=x
N_seq <- 1:50
err_x <- L2_error(S_N_x, function(x) x, N_seq, x_fun = S_N_x)

plot(N_seq, err_x, type = "o", pch = 16, cex = 0.6,
     xlab = "N", ylab = expression(group("|", S[N]-f, "|")[L^2]),
     main = expression("L2 error vs N (f(x)=x)"))
```




```r
# f(x) = sin(2x)

# Graph 1: S_N(x) vs f(x) = sin(2x)
plot(x_plot, sin(2*x_plot), type = "l", lwd = 2, col = "black",
     xlab = "x", ylab = "y", main = expression("f(x)=sin(2x):"~S[N](x)~"vs f(x)"))

# Adding the Fourier series partial sums
for (N in c(2, 5, 10)) {
  lines(x_plot, S_N_sin2x(x_plot, N), col = N, lty = 2)
}

legend("topright", legend = c("f(x)", "S_N (N>=2)"),
       col = c("black", 2), lty = c(1, 2), cex = 0.8)
```




```r
# Graph 2: L2 error vs N for f(x)=sin(2x)
N_seq2 <- 1:20
err_sin2x <- L2_error(S_N_sin2x, function(x) sin(2*x), N_seq2, x_fun = S_N_sin2x)

plot(N_seq2, err_sin2x, type = "o", pch = 16, cex = 0.6,
     xlab = "N", ylab = expression(group("|", S[N]-f, "|")[L^2]),
     main = expression("L2 error vs N (f(x)=sin(2x))"))
```



Observations (a):

- **f(x) = x:** Gibbs Phenomenon: As N increases, the partial sums $S_N(x)$ track the linear slope of $f(x)$ more closely in the center of the interval. However, because the periodic extension of $f(x) = x$ has a jump discontinuity at the boundaries (0 and 2π), prominent oscillations persist near these edges. The plot of the L^2 error shows a steady, gradual decrease as N increases. The error does not drop to zero quickly because the higher-frequency components are necessary to attempt to resolve the sharp discontinuity at the boundary.

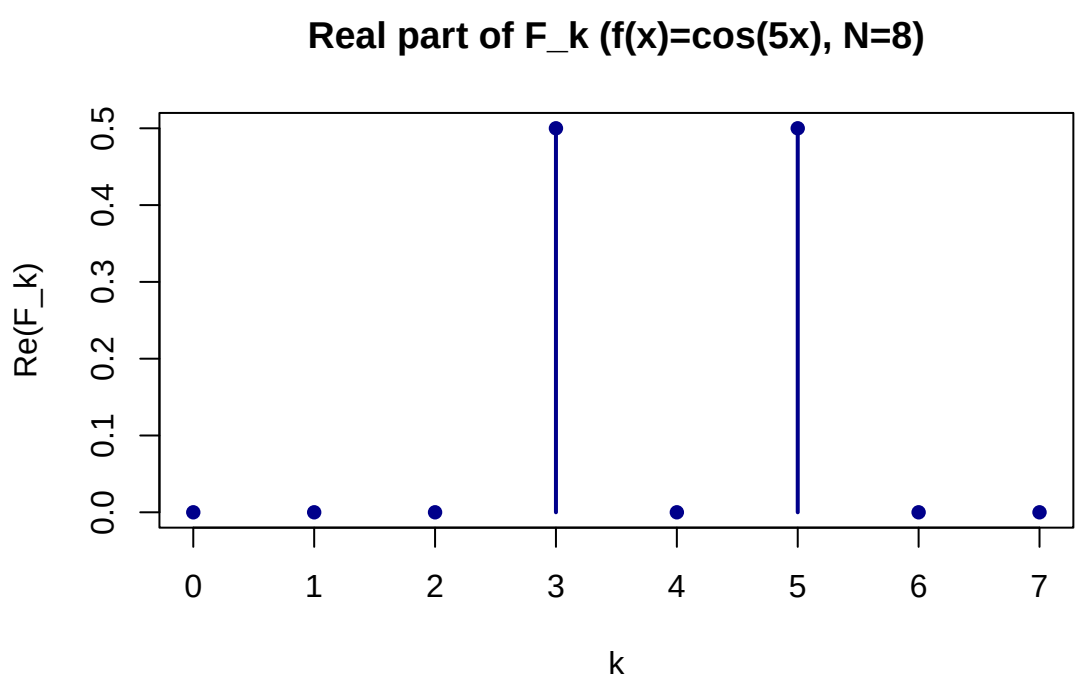
- **f(x) = sin(2x):** For $N \geq 2$, the approximation $S_N(x)$ is visually indistinguishable from the true function $f(x)$. Since $\sin(2x)$ is itself one of the basis functions used in the Fourier series, the approximation becomes exact as soon as that frequency is included in the sum. The L^2 error plot confirms this "perfect fit". The error is high for $N = 1$, but it drops immediately to zero for $N = 2$ and remains at zero for all subsequent values of N. This demonstrates that finite Fourier series can perfectly reconstruct functions composed of a finite number of sinusoids.

(b) DFT Implementation (Without FFT)

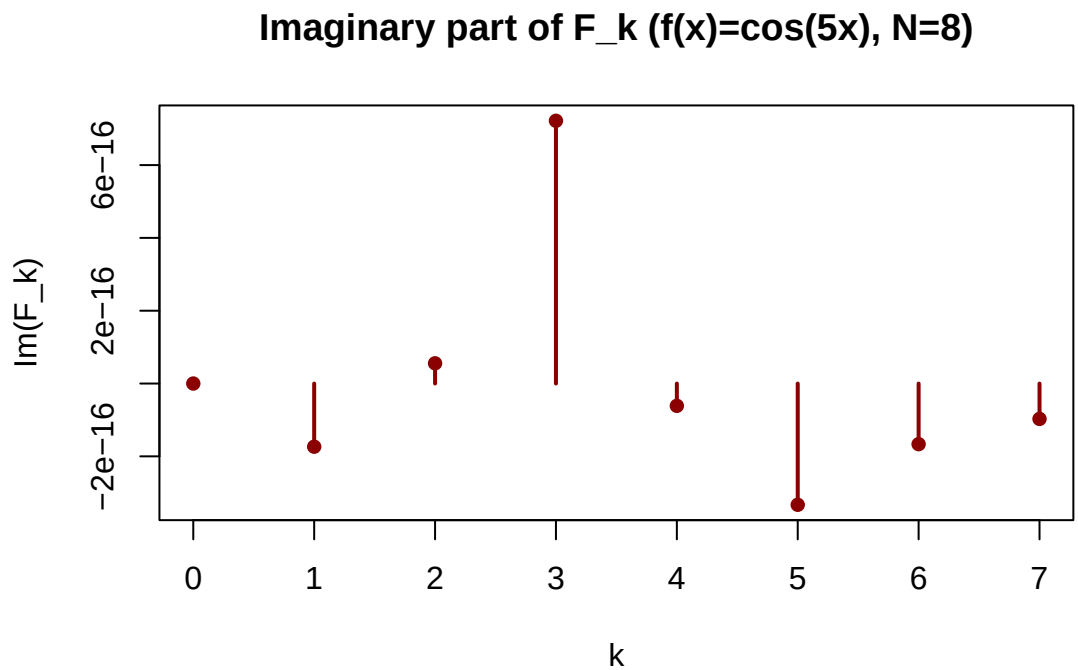
We compute the DFT coefficients $F_k = \frac{1}{N}\sum_{j=0}^{N-1} f(x_j)e^{-ikx_j}$ with $x_j = \frac{2\pi j}{N}$, without using built-in FFT. We choose a function, compute F_k, and plot real and imaginary parts to observe aliasing.

```
# DFT: F_k = (1/N) sum_{j=0}^{N-1} f(x_j) exp(-1i * k * x_j), x_j = 2*pi*j/N
dft_manual <- function(f_vals) {
  N <- length(f_vals)
  k_vals <- 0:(N - 1)
  F_k <- complex(length(k_vals))
  for (k in k_vals) {
    F_k[k + 1] <- 0
```

```r
    for (j in 0:(N - 1)) {
      x_j <- 2 * pi * j / N
      F_k[k + 1] <- F_k[k + 1] + f_vals[j + 1] * exp(-1i * k * x_j)
    }
    F_k[k + 1] <- F_k[k + 1] / N
  }
  F_k
}

# Example: f(x) = sin(2x) sampled at N points. For N small, aliasing occurs.
# Another example: f(x) = cos(5x) with N = 8 (few samples) to see aliasing
N_small <- 8
x_nodes <- (0:(N_small - 1)) * (2 * pi / N_small)
f_cos5 <- cos(5 * x_nodes)  # frequency 5 with only 8 points
F_cos5 <- dft_manual(f_cos5)

k_plot <- 0:(N_small - 1)
plot(k_plot, Re(F_cos5), type = "h", lwd = 2, col = "darkblue",
     xlab = "k", ylab = "Re(F_k)", main = "Real part of F_k (f(x)=cos(5x), N=8)")
points(k_plot, Re(F_cos5), pch = 16, col = "darkblue")
```




```r
plot(k_plot, Im(F_cos5), type = "h", lwd = 2, col = "darkred",
     xlab = "k", ylab = "Im(F_k)", main = "Imaginary part of F_k (f(x)=cos(5x), N=8)")
points(k_plot, Im(F_cos5), pch = 16, col = "darkred")
```



Aliasing explanation (b): We use $f(x) = \cos(5x)$ with only $N = 8$ samples. The true frequency is 5, but with 8 samples the DFT only has indices $k = 0, \ldots, 7$. For a real signal, $F_{N-k} = \overline{F_k}$, so energy at high frequency "folds back." Here $\cos(5x) = \frac{1}{2}(e^{i5x} + e^{-i5x})$; with period 2π, mode 5 is equivalent to mode $5 - 8 = -3$, so we see nonzero coefficients at $k = 3$ and $k = 5$ (and by symmetry at $k = 8 - 3 = 5$). So the high frequency (5) is aliased to a lower index. This illustrates that if we sample below the Nyquist rate (here we need at least 10 points to resolve frequency 5), the DFT coefficients do not uniquely represent the original continuous frequency.

(c) Trigonometric Interpolation Using DFT Coefficients

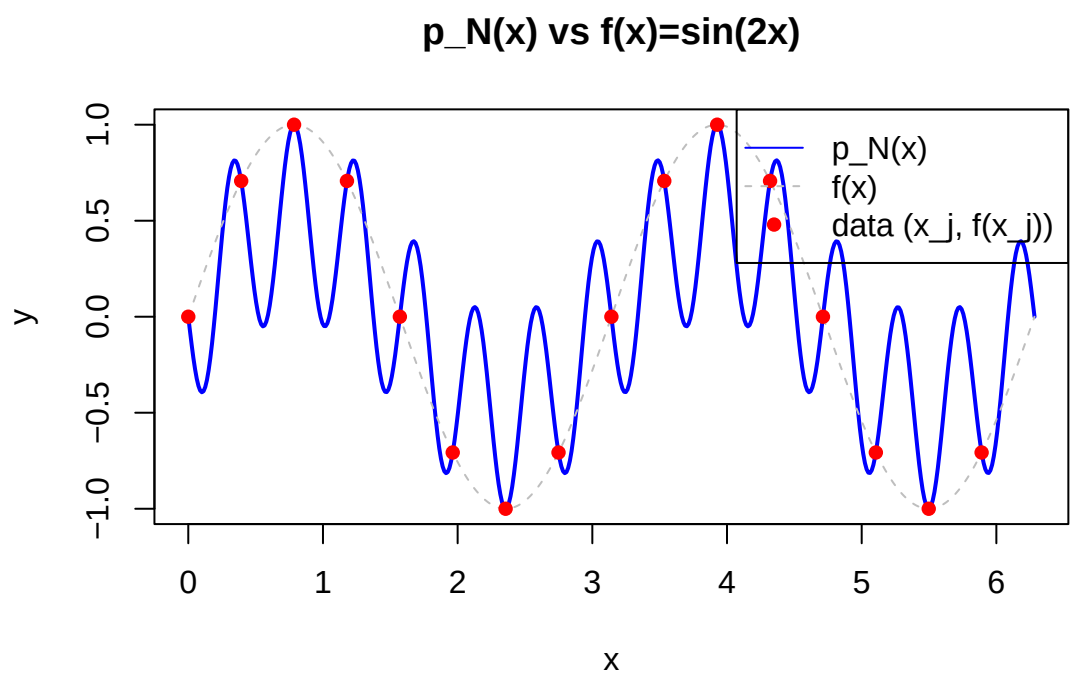
Using the F_k from part (b) for $f(x) = \sin(2x)$, we form the trigonometric interpolant $p_N(x) = \sum_{k=0}^{N-1} F_k e^{ikx}$ and verify that it interpolates the data $(x_j, f(x_j))$, and plot the interpolation error $|p_N(x) - f(x)|$.

```
# For f(x) = sin(2x): sample at x_j = 2*pi*j/N, compute F_k, then p_N(x) = sum_k F_k exp(i k x)
N <- 16
x_j <- (0:(N - 1)) * (2 * pi / N)
f_vals <- sin(2 * x_j)
F_k <- dft_manual(f_vals)

# Trigonometric interpolant: p_N(x) = sum_{k=0}^{N-1} F_k exp(i k x)
p_N <- function(x, F_k) {
  N <- length(F_k)
  out <- 0 + 0i
  for (k in 0:(N - 1)) {
    out <- out + F_k[k + 1] * exp(1i * k * x)
  }
```

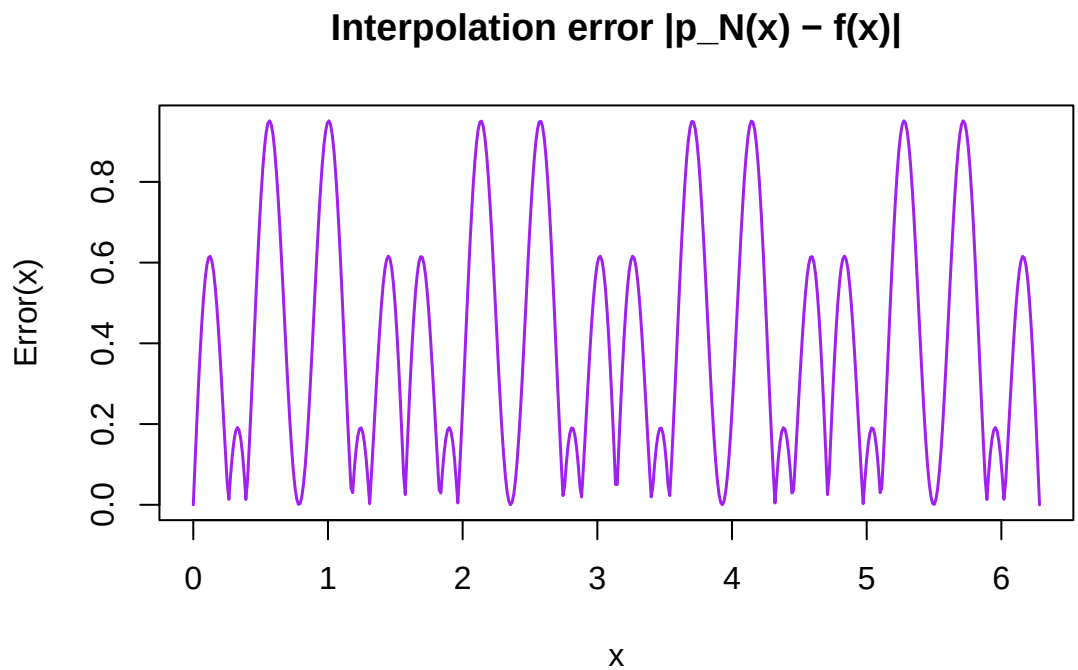
```r
  Re(out)  # for real data, imaginary part is numerical noise
}

# Verify at nodes: p_N(x_j) should equal f(x_j)
p_at_nodes <- numeric(N)
for (j in 1:N) p_at_nodes[j] <- p_N(x_j[j], F_k)
cat("Max interpolation error at nodes:", max(abs(p_at_nodes - f_vals)), "\n")
```

```
## Max interpolation error at nodes: 4.884981e-15
```

```r
# Plot interpolation error over [0, 2*pi]
x_fine <- seq(0, 2*pi, length.out = 500)
f_true <- sin(2 * x_fine)
p_fine <- sapply(x_fine, function(x) p_N(x, F_k))
err_interp <- abs(p_fine - f_true)

plot(x_fine, p_fine, type = "l", col = "blue", lwd = 2,
     xlab = "x", ylab = "y", main = "p_N(x) vs f(x)=sin(2x)")
lines(x_fine, f_true, col = "gray", lty = 2, lwd = 1)
points(x_j, f_vals, pch = 16, col = "red")
legend("topright", legend = c("p_N(x)", "f(x)", "data (x_j, f(x_j))"),
       col = c("blue", "gray", "red"), lty = c(1, 2, NA), pch = c(NA, NA, 16))
```




```r
plot(x_fine, err_interp, type = "l", col = "purple", lwd = 1.5,
     xlab = "x", ylab = "Error(x)", main = "Interpolation error |p_N(x) - f(x)|")
```



Interpretation (c): The trigonometric interpolant $p_N(x)$ achieves perfect accuracy at the discrete nodes, with a maximum error of 4.88×10^{-15} reflecting the limits of machine precision. While $p_N(x)$ is a smooth 2π-periodic polynomial that correctly interpolates the data points, significant error occurs between nodes because the summation uses only high positive frequencies ($k = 0$ to $N - 1$) rather than centering them around zero. In this case, with $N = 16$, the error $\|p_N(x) - f(x)\|$ remains zero at the nodes but oscillates in between due to the lack of negative frequency components necessary to reconstruct the smooth $\sin(2x)$ signal. This demonstrates that while the DFT ensures exactness at sampled points, the representation of the function between those points depends heavily on whether the sampling rate is sufficient to resolve the underlying frequencies without aliasing.